BALANCING NEEDS, EFFICIENCY, AND FUNCTIONALITY IN THE PROVISION OF MODELING SOFTWARE: A PERSPECTIVE OF THE NIH WINSAAM PROJECT.

Peter Greif,[1] Meryl Wastney,[2] Oscar Linares,[3] Ray Boston[4]

[1]Laboratory of Experimental and Computational Biology
National Institutes of Health, Washington DC
[2]Department of Neonatology
Georgetown University Medical Center, Washington, DC
[3]Department of Geriatrics
University of Michigan Medical School, Ann Arbor, MI
[4]Clinical Studies Department, New Bolton Center
University of Pennsylvania

ABSTRACT

The development of new software or the refinement of existing software for new operating environments each calls for judicious balancing. On the one hand, we strive for simplicity, predictability, and operational protection as it is well recognized that software with these attributes will attract an audience of satisfied users. But, on the other hand, these attributes do not conjure a sense of power, efficiency, or flexibility, and these other properties are also appreciated by users, albeit a somewhat different group of users. The goal is to achieve a blend which isolates critical functionality, flexible control, and user support while meeting the needs of the broadest collection of serious users. In this chapter, we discuss the issues impacting the migration of SAAM to the Windows environment, the NIH WinSAAM Project, and we outline the steps taken to ensure its feasibility. In addition, we describe a new paradigm for software development and use which ensures the durability of the software for modeling.

INTRODUCTION

Mathematical models have been used extensively as tools for nutritional research. Indeed, it is unlikely that any serious examination of a nutritional subsystem could be advanced without recourse to current related models. The reason for this is clear: the systems explored involve multiple tissues and organs, multiple metabolites and nutrients,

A. J. Clifford et al. (eds.), *Mathematical Modeling in Experimental Nutrition*
© Springer Science+Business Media New York 1998

and multiple processes and mechanisms, and it is not possible to explore the complex effects involved here without systematically formulating the nature of each from a mathematical perspective.

Nutritional models have been developed to describe whole-animal level functions (Baldwin, 1995), organ levels functions (Dijkstra, 1993), tissue level functions (Gill, 1984), and single substrate level functions (Faichney, 1985). The have also been used for processes embracing digestion (France, 1982), absorption (Pettigrew, 1992), metabolism (Hanigan, 1994), and production (France, 1984). Indeed, models in the nutritional area have even been developed at the cellular level to explain production consequences of metabolism there (Neal, 1984).

Thanks to the work of Thornley (1991), France (1984), and Baldwin (1987) on the one hand, and Berman (1983), Jacquez (1996), and Cobelli (1983) on the other, fairly clear paths now exist for the development of models. The first group has refined the so-called 'transactional' perspective of metabolic systems in which only the exposed or limiting aspects of metabolic pathways or digestive processes are incorporated into models. From this integrated perspective, they adopt the position that if an adequate representation of the fundamental processes (transactions) of subsystems are evident, then integration across subsystems should lead to realistic system-wide response predictions. To help with their realistic (effective) portrayal of subsystems (transactions), they have developed a number of ingenious mathematical representations of metabolic steps including substrate limited processes, allosteric binding (also referred to as threshold processes), various forms of inhibition, and hormonal sensitization. A number of large systems involving 17 (Dijkstra, 1992) to approximately 60 (Baldwin, 1995) state variables have been modeled in this way.

Large-scale modeling, particularly transactionally-based, large-scale modeling, needs access to the results of investigations which define the properties of the rate-limiting processes in order to incorporate them into the representation of the sub-systems... The results of what might be referred to as micro-scale modeling. It is this area of modeling which has benefited from the work of the second group of investigators. They have systematically refined approaches which permit the accurate characterization of sub-system processes. These processes include model decoupling, model reduction, identifiability, minimal modeling, and constrained modeling (see for example, Boston, 1984; Jacquez, 1996; Cobelli, 1983). Used appropriately, these techniques can greatly enhance the quality of a system investigation, especially where the nature of this data masks simple direct data fitting and interpretation.

To enable these strategies to yield their full potential, the tools, techniques, and methodological efforts refined by both groups have had to be matched with powerful computer software. Initially, this was provided by SAAM (1978) software, where an ingenious approach to model-fitting and data analysis enabled direct fitting of complex systems to data. This software was originally developed in the sixties for the analysis of tracer data, but rapidly evolved to the tool of choice for micro-scale modeling groups. Around the same time, SAAM was evolving, a large-scale system simulator, CSMP (Fugazi, 74), was also taking shape which provided comprehensive computer support for investigators involved with the development of vary large models. In the seventies, CSMP was replaced by ACSL (1992), and this package has clearly emerged as the tool of choice for modelers building large systems.

Since the eighties, we have witnessed, on the one hand transformations, and enhancements to the older modeling software, and on the other hand, the emergence of a series of new 'look and feel' software with the attendant buttons, windows, icons, and menu-based services we now currently expect of computer software. In this latter group we include Stella, Scientist, and SAAM II as outstanding examples of how modeling functions can be incorporated into the 'quasi-mode-less' application setting of typical Windows

programs. The first group of Windows software, the ported software, has had to address many more and complex issues in regard to its target setting than those worked into this setting from the ground up. These included such critical issues as whether to re-deploy function access by capitalizing on the built-in tools of the Windows environment, or to displace command lines and command-based processing flow in favor of buttons and menu icons in order to deliver essentially the same operations.

It seems at first flush unconscionable that we could ask these types of questions, for there is no doubt that those without prior (and perhaps extensive) exposure to mode-less software would be lost without the attendant widgets to guide them through their agendas. But by the same token, skilled and efficient software users (a stage to which we almost all eventually aspire) who are comfortable with the CLI (command line interface) paradigm, feel cheated and impeded by this sometimes cumbersome and always less flexible form of operational control. This chapter will explore these issues and reconcile our approaches and implementations as we recount efforts to port Consam to the Windows setting ...The NIH WinSAAM project.

METHODS

Because of the difference between this chapter and the others in this series, we will describe our approach in terms of relating the Methods, Results, and Conclusion sections of our work. In the Methods section and under appropriate sub-headings, we will present the WinSAAM Design Objectives and Design Basis, as well as the software architecture. In the Results section, we will present some implementation details and outline our major new directions for modeling software per se. Finally, in our Conclusion, we will describe the state of our current and planned rates of progress and how interested groups might participate in the WinSAAM project.

Design Objectives

Whereas higher aspirations such as 'goals' might be conceived as the target of a software refinement effort, Press (1972) has cautioned against this. Indeed, he stresses that the outcome, or production, needs to be evaluated against a concrete set of criteria and hence endorses 'objectives' for this purpose. We have isolated twelve objectives of our approach that ought to be evident in and through the operational style of our final product.

Incorporate the total functionality of SAAM and Consam into WinSAAM.

We believe that if SAAM and Consam are to become obsolete through WinSAAM, then the SAAM user-community needs access to a software tool with at least the combined capabilities of the former versions. Thus, WinSAAM will functions as a superset of the functionality of the earlier versions. In some respects WinSAAM will incorporate additional functionality, however, at this stage extended functionality will primarily relate to access to Windows tools to manage processing more flexibly in the Windows setting.

Preserve the operational flow of Consam.

Because of the wide interest Consam has attracted to SAAM-style modeling, and because of the need to embrace new WinSAAM users in a familiar and productive fashion, WinSAAM will support a 'terminal' window to provide instant access to the processing capabilities and operational flow of Consam. Indeed, in our estimation a former Consam user will be immediately able to run WinSAAM in a productive way.

Integrate the 'look and feel' of Windows products.

If the target operating environment of WinSAAM is Windows, than clearly it would be most inappropriate not to incorporate a Windows-type 'look and feel' into the system. Here, for example, we intend that file management, text editing support, clipboard access, and allied attendant Windows widgets and tools will be routinely available and efficiently deployable while running the system.

Built on a third-party component basis.

A clear advantage in developing a product in the Windows environment is that one immediately gets access to the vast array of 'software add-ins.' These are special purpose tools (editors, spreadsheets, graphics tools, plotting tools, etc.) which, upon purchase, can be smoothly incorporated into a product on a component-by-component basis. With the savings offered to us by adopting this strategy, we decided that as much as possible of the new interactive tools for WinSAAM would be comprised of third-party components.

Prototype incrementally.

Because of the limited man-power of the WinSAAM project team, we proposed an incremental prototyping paradigm for development. As each new component of the software is foreshadowed for inclusion, an iterative implementation and refinement approach was adopted (see also 'design basis' below). In this setting, once a component functions properly and integrates smoothly with the other components, it is considered a part of the system. Regular integration testing of the product, at any stage, can be undertaken to confirm its reliability and processing flow.

Design for an open style of operation.

Since the first release of Consam in 1978, users have enjoyed considerable freedom in controlling their modeling sessions. Indeed, what we witnessed was an attempt to create a fully-protected, command-based, mode-less environment. We have set ourselves this objective again with WinSAAM. That is, as opposed to tightly controlling and synchronizing user actions, we will attempt to cater for a more 'free wheeling' (mode-less), but protected operational flow.

Design for extensibility.

In keeping with the development deployment of third-party software and, at the same time, recognizing the valuable contributions users and other software developers may offer, we propose that WinSAAM demonstrate and capitalize on two key areas of extensibility: modeling libraries and extended functionality. We propose a means whereby focus groups in various areas of modeling (e.g. lipid metabolism, or glucose/insulin kinetics) can be registered to enable them to provide model 'add-ins.' Other WinSAAM users will then be able to subsequently access and use these models. In addition, we intend to facilitate the functional extension of WinSAAM via 'DLLs' (dynamic linking libraries) in much the same fashion as is done by Scientist. As new processing algorithms are developed, we can explore their incorporation directly into WinSAAM without actually modifying the underlying WinSAAM code.

Accommodate communication with other software.

One of the great problems associated with developing software is defining the extent of its functionality. For example, software providing data-fitting capabilities could potentially be extended to allow, at one end, data preparation support, and, at the other end, statistical tools for analysis and review of the model fit. However with the powerful data management and statistical processing software available, simply providing access to these

from the data-fitting software may be enough to address most needs. We intend to implement the most flexible and most common array of data transportation tools into WinSAAM to enable user-preferred access to pre-processing and post-processing software.

Accommodation of an efficient help facility.

As was common to its time, SAAM was poorly documented and difficult to use. Consam possessed some of these traits as well, largely because of its close relationship to SAAM. With the pervasive availability of powerful 'help generators,' we intend that WinSAAM will be comprehensively and efficiently documented for user convenience.

Ensure that WinSAAM is consistent, reliable, and efficient.

The fact that, on the one hand, WinSAAM is built around the well-worn operational paradigm of SAAM and Consam and, on the other hand, its kernel (SAAM) has been subject to many years of rigorous testing and evaluation, gives us a competitive edge in assuring the overall robustness of the new WinSAAM software. Nevertheless, we intend to deploy all the same quality control checks with WinSAAM that we have submitted both SAAM and Consam to in the past. We will also ensure that WinSAAM's flow and mode-less operational style are efficient in delivering user needs in a consistent style across the operational interface.

Distribute application activities amongst the user community.

As we have mentioned, application of the new software will be driven through the incorporation of 'domain' libraries. Users skilled in particular domains will be able to be registered as library domain contributors; the incorporation of their library domain into the WinSAAM application library will provide a vehicle for the robust extention of the routine application of the software. This will expedite evolution of the software application base and, at the same time, ensure that topical applications are routinely available to WinSAAM users.

Preservation of the SAAM kernel; architecture, and organization.

To ensure that all of SAAM's functionality is available in WinSAAM, and to maintain momentum with WinSAAM delivery, the SAAM kernel as incorporated in Consam will, in its current form, provide the processing heart of WinSAAM. Whereas restructuring SAAM's architecture may eventually expedite maintenance and enhancement efforts for WinSAAM, the net yields to the user community from our diverting efforts to this purpose were not justified at this stage. In fact, as seen above, maintenance and enhancement efforts, which give the highest returns for effort, have been identified as the critical modules we will be adding and linking into the SAAM processing environment.

Design Basis

As each step in the development of WinSAAM software is negotiated, it is carefully evaluated against the following criteria:

- Does it fulfill or comply with our design objectives?
- Is there a preferred implementation approach which may facilitate its use or access?

Based on the consequences of exploring these issues, a prototypical implementation of the step (e.g., the incorporation of a service or refinement of a display) is created. After some routine modeling analysis to exercise the new facility, aspects are reviewed not only against the above criteria but also in regard to the ease with which they fit into the overall WinSAAM setting.

It is hoped that by using β-testing and further iterative refinement, faster and smoother closure on the first release version will be achieved.

WinSAAM Architecture

In Figure 1, we present an architectural perspective of WinSAAM. We see that there are essentially four classes of software components: locally-developed components which largely collect and coordinate related WinSAAM functions; third-party 'add-ins' or component software, each providing a complete aspect of interface processing needs; library modules, including a base library and an integrated set of domain-specific libraries; and a set of DLLs, including the SAAM/Consam system, and a set of functional enhancements.

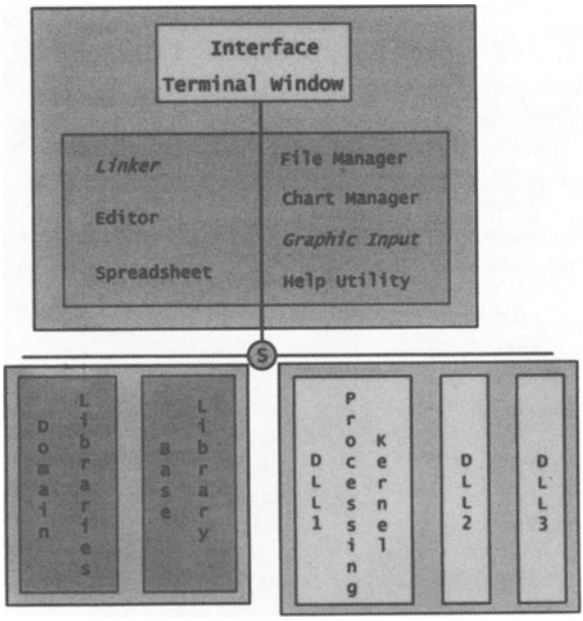


Figure 1. The architectural organization of WinSAAM. There are four types of functional modules incorporated into three activity domains: (1) in the interface, the terminal window (with a supervisory role as indicated), and the linker, the editor, the spreadsheet manager, the file manager, the chart manager, the graphical input manager, and the help utility; (2) in the library area, the base library, and the domain libraries; and (3) in the processing area, the kernel, and an array of client dlls. Those areas currently underway are italicized.

To clarify the functional capabilities and processing integration of WinSAAM, we present the purpose and capabilities of its interface components in Table 1.

Table 1. WinSAAM interface objects and their functions.

Interface Object	Functions
Terminal Window	Terminal input Line-oriented text editor CLI-style processing control
.Editor	Graphic text management Tex input editing (graphic) Text output editing (graphic)
Spreadsheet	Tabular-oriented input Tabular-oriented processing Tabular-oriented output Tabular-oriented formatting Tabular-oriented communication
File Manager	Coordinate user files Facilitate file access Check integrity of user files
Chart Manager	Plotting results Configuring plots Chart-based communication
Linker	Assemble input components Edit project specification Manage projects Facilitate project associations
Graphics Module	Graphical model input Graphical-lexical model translation

RESULTS

Aspects of the implementation of WinSAAM can be gleaned from Figures 2 to 6, which present screen dumps of various states of WinSAAM processing.

Figure 2 shows the so-called 'terminal' or introductory window, with the familiar lexical display of a pre-assembled WinSAAM input. Also shown is the simple menu scheme which provides access to the integrated array of support tools embracing the file manager, the spreadsheet analyzer, and the editor.

Figure 3 shows the integrated graphical text editor provided to enable users to rapidly edit lexical (text) information including WinSAAM input files, components (or segments) of input, and familiar full processing logs.

In Figure 4, we show the datasheet of our notebook-oriented spreadsheet output. Sheets are compiled into pages labeled 'data,' 'parameters,' 'covariance,' and 'partials' to provide the user access to all processing results available from system solving and iterative parameter adjustment. Results sheets can be easily reformatted, transformed, saved as EXCEL compatible inputs, or captured for post-processing analysis in other statistical and related software.

In Figure 5, we present a plot of some WinSAAM processing results captured in PowerPoint from WinSAAM's charting subsystem, via the clipboard. With the capacity to scale, zoom, and rotate plots, as well as label and algorithmically modify axes, this subsystem fits the new demands of the WinSAAM environment extremely well. Indeed, this utility has been targeted to provide all the basic support needed for users to take WinSAAM results from a research investigation level to publication quality output.

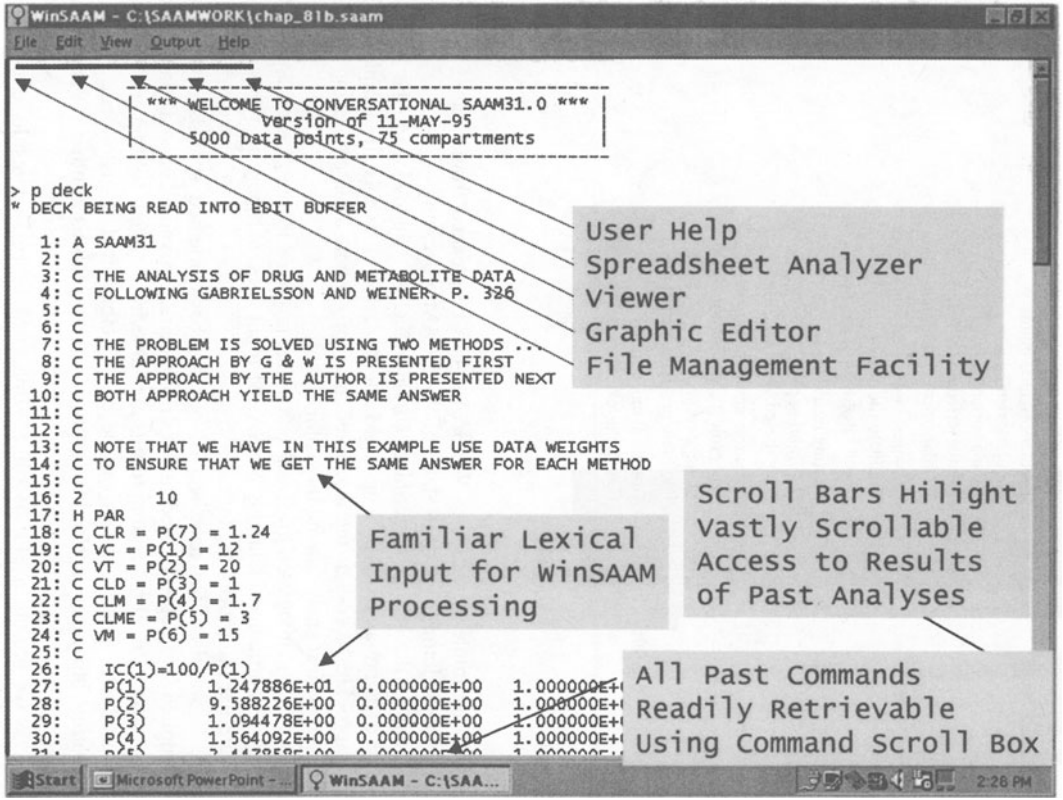


Figure 2. A screen dump of the WinSAAM terminal window showing aspects of a modeling session captured here and the operational flow supported from this point.

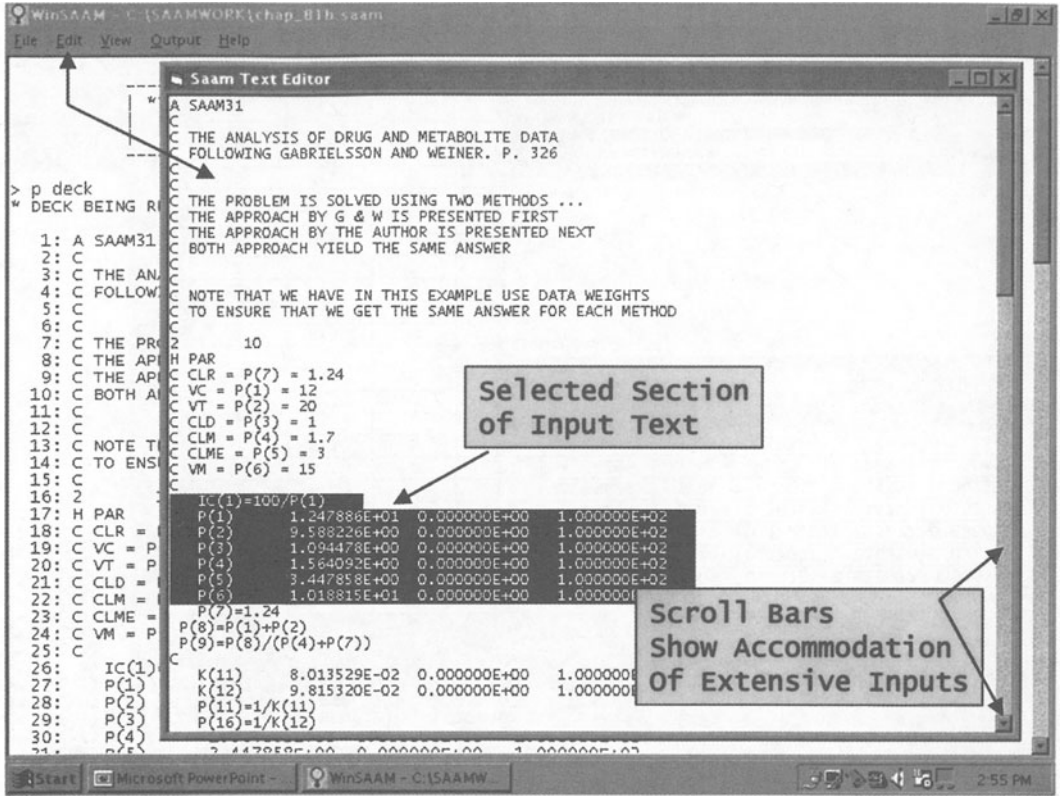
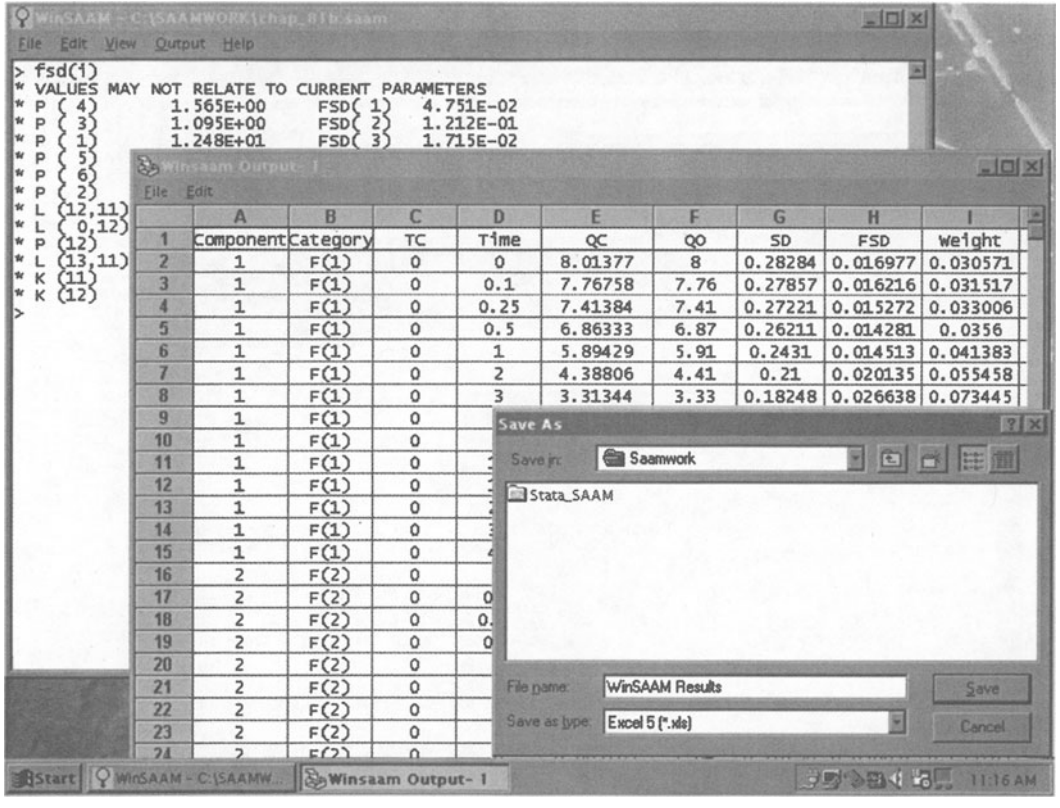


Figure 3. A screen dump showing the steps leading to WinSAAM's graphic text editor as well as some interaction with this editor.



	A	B	C	D	E	F	G	H	I
1	Component	Category	TC	Time	QC	QO	SD	FSD	weight
2	1	F(1)	0	0	8.01377	8	0.28284	0.016977	0.030571
3	1	F(1)	0	0.1	7.76758	7.76	0.27857	0.016216	0.031517
4	1	F(1)	0	0.25	7.41384	7.41	0.27221	0.015272	0.033006
5	1	F(1)	0	0.5	6.86333	6.87	0.26211	0.014281	0.0356
6	1	F(1)	0	1	5.89429	5.91	0.2431	0.014513	0.041383
7	1	F(1)	0	2	4.38806	4.41	0.21	0.020135	0.055458
8	1	F(1)	0	3	3.31344	3.33	0.18248	0.026638	0.073445
9	1	F(1)	0						
10	1	F(1)	0						
11	1	F(1)	0						
12	1	F(1)	0						
13	1	F(1)	0						
14	1	F(1)	0						
15	1	F(1)	0						
16	2	F(2)	0						
17	2	F(2)	0						
18	2	F(2)	0						
19	2	F(2)	0						
20	2	F(2)	0						
21	2	F(2)	0						
22	2	F(2)	0						
23	2	F(2)	0						
24	2	F(2)	0						

Figure 4. A screen dump displaying the sequence of steps from the WinSAAM terminal window to the preparation of the WinSAAM results for output as an EXCEL 5 spreadsheet. The intermediate spreadsheet provides for the highly flexible re-organization of the WinSAAM processing results as needed prior to transfer of them to an EXCEL file, and presumably to EXCEL for further analysis.

Figures 5A-5C. This series of three figures illustrate the flexibility of the WinSAAM graphics system. In each case, the WinSAAM plot was captured in the clipboard as a Windows metafile and transferred directly to PowerPoint for improvement, color enhancement, or more comprehensive annotation.

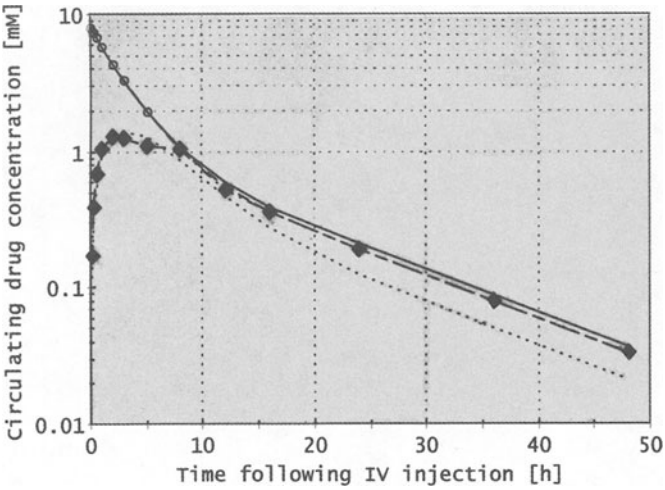


Figure 5A: A standard two-response plot in which we have also joined the data points (long and short dashes) to help clarify the consistency between the observed and fitted points. Joining the observation set in a data series is very easily accomplished with the WinSAAM charting system.

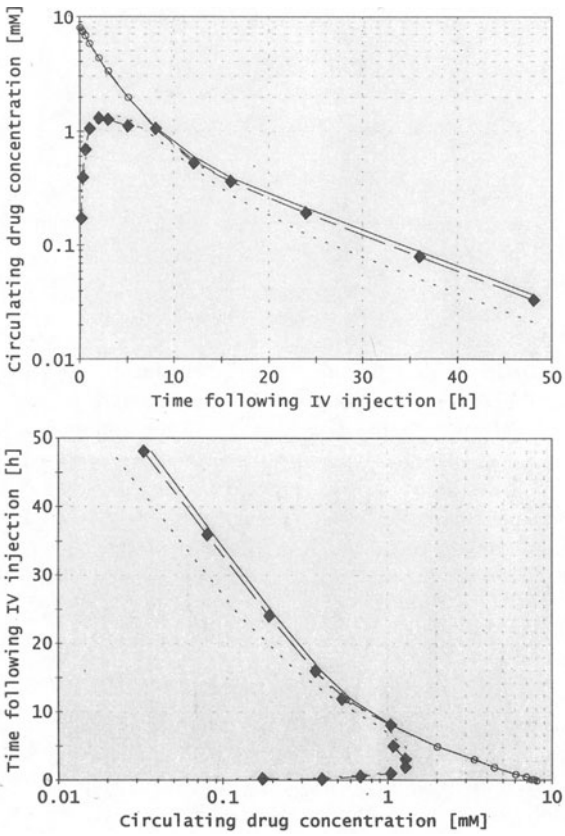


Figure 5B: The above plotted first as per Fig. 5A (upper plot), and secondly plotted on its side (lower plot). Changing the x-y assignment of plots often helps clarify the nature of a relationship between two variables. It requires no more than a one button activation to accomplish the role reversal in WinSAAM's charting system.

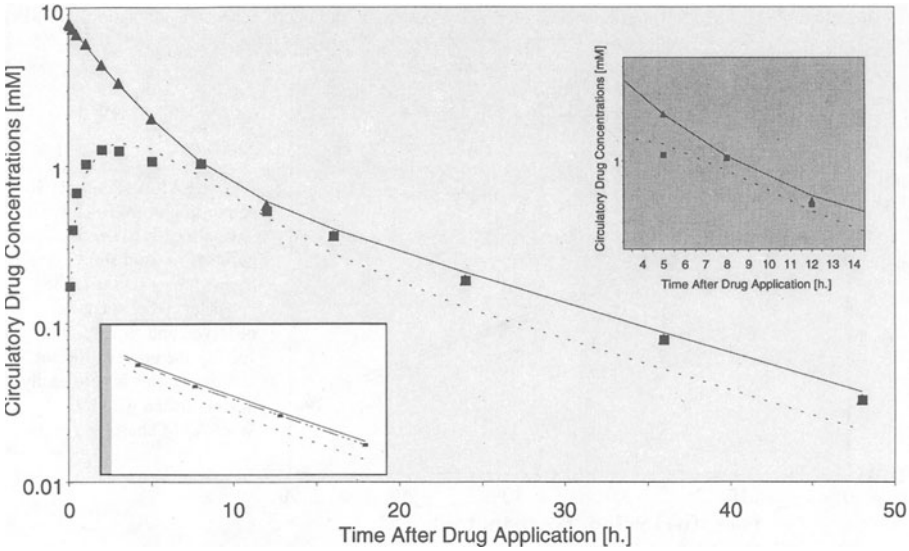


Figure 5C: The same plot as in Fig. 5A, but in addition we have here incorporated detailed views of (a) the values near the peak of the large plot (upper inset), and (b) values near the tail of the large plot (lower inset). It requires no more that a simple drag to accomplish either zooming (upper inset) or re-scaling (lower inset), and restoring the original plot (foreground plot) involves only a single key press. Features such as this can be of enormous assistance to investigators.

Finally, Figure 6 illustrates the ease with which WinSAAM results are made available for further statistical or related examinations using a typical general purpose statistical package. Specifically, in the top left corner of the figure, we see the WinSAAM data spreadsheet from which the results were exported. Next, as a spreadsheet overlaying the WinSAAM spreadsheet, we see STATA's data entry spreadsheet with the WinSAAM results automatically transferred into it. In the foreground, we see the typical STATA terminal interface with a **de**scription (STATA command) of the WinSAAM data, followed by a strategic **su**mmary (STATA command) of the same data. Finally, in the bottom left corner of the figure we see a STATA plot of the observed and calculated values exported from WinSAAM to STATA. This entire process involved no more than four button activations and the invocation of a couple of simple STATA commands. A particular feature of the process is the automatic naming of WinSAAM variables as the data is compiled into the STATA spreadsheet.

NEW DIRECTIONS

The movement of the SAAM/Consam software to Windows has enabled us to explore and refine a number of new directions. While these may pertain most particularly to SAAM and Consam, to some extent they may apply to modeling methodology per se, specifically with regard to how software may better meet the needs of investigators.

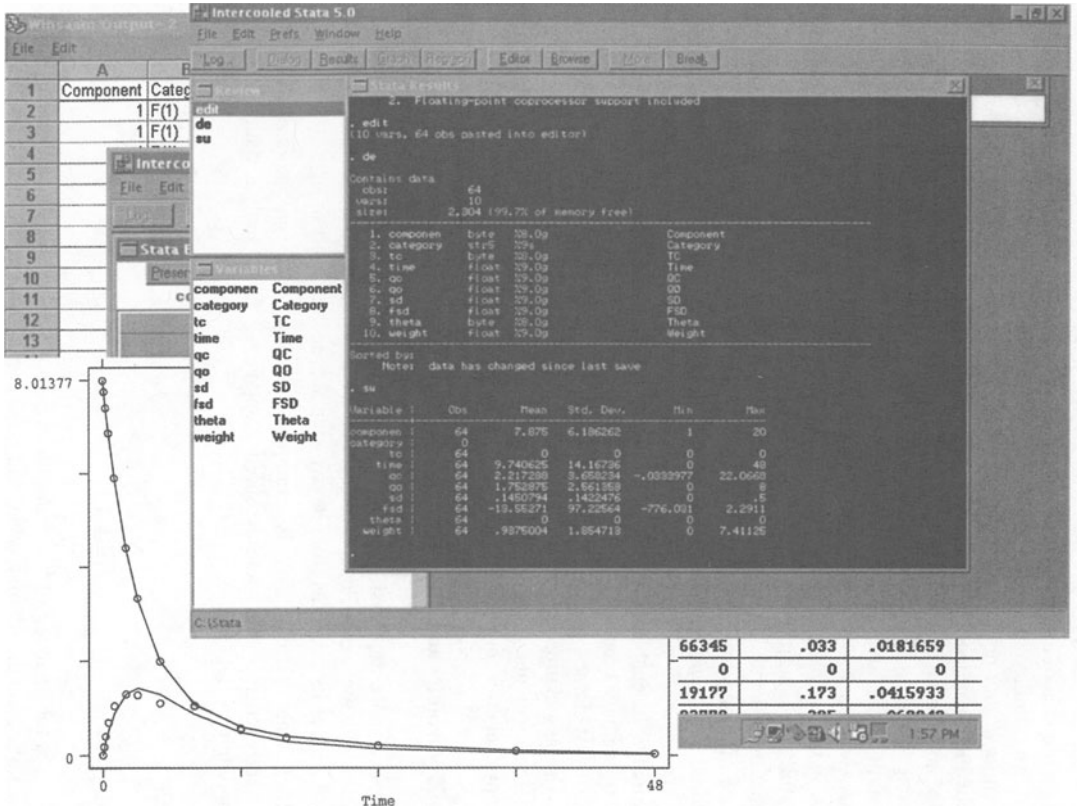


Figure 6. The steps associated with exporting WinSAAM results to an advanced statistical package (STATA) for further analysis. See text for details.

Input Units, the Modeling Project, and the Linker

During the redesign of Consam's file management facility for WinSAAM, it became clear to us that the opportunity had now arisen in which we could radically re-think the entire method by which objects can be assembled and linked in preparation for processing. Specifically, whereas inputs to Consam were historically made on a 'job' basis in which a single input data set was combined with a single model for a single analysis, we could now entertain the idea of alternate data sets and models from different bases linked for analysis. Indeed, we stumbled on the idea of *Input Units*, *Modeling Projects*, and the *Linker*.

An *input unit* represents the atomic or minimal entity from which information can be complied into a modeling investigation. This could be the data associated with a particular stage of an experiment, or it could be a model acquired from the internet considered worthy of testing against a local set of data (which would also be considered an input unit). An input unit could also be as large as the total set of information needed for a single WinSAAM run, i.e. a self-contained unit, or a saved modeling project.

A *modeling project* constitutes all the files essential for a modeling session. Thus, more fully, the project comprises essential input units, as well as essential file-based information generated in what we may refer to as the 'instance of a modeling session.' An instance of a modeling session represents all the external resources needed (file names and locations, file assignments and associations, for example) for WinSAAM interaction to take place in association with a particular modeling project.

Finally, to enable a better integration of input units and management of modeling projects, we have created the notion of a *linker*. In addition to displaying linkable items in their array of classes, in preparation for project creation, the linker will facilitate both their class-wise association (i.e. naming or identifying global functions within the model units, portraying segments within the data units or within time segments of the data units), and their physical collection for project processing.

Derived Variables and Finesse Fitting

Conceivably the most efficient approach for processing data from several identical studies is with the use of Zeger's (1994) derived variable analysis, or 'finesse' fitting. This method derives its strengths from its two-step approach to analysis viz:

Step One: Each study is examined to isolate a *single* model structure which consistently characterizes all measured responses. Each study's data units are fitted appropriately to yield

$$\begin{aligned} \mu_i &= \mu + r_i \\ r_i &= a_i r \qquad (E(a_i) = 1) \end{aligned} \tag{1}$$

where μ_i is the estimate of μ for the *i*th individual; μ, the population mean value; r_i, the estimate of r from the *i*th individual; and r, the population covariance.

Step Two: μ and r are estimated using either the method of maximum likelihood or its restricted version (to avoid bias).

The strengths of this two-step approach are:

- It commits the modeler to preserving the consistency between the target population model and each specific study.
- It is extremely flexible and virtually limitlessly extensible.
- It easily accommodates a cumulative development of the covariance base.

- It can readily be modified to incorporate Bayesian methods concerning future or prospective applications of the model.
- It leads to immediate isolation of aberrant individuals.

A limitation of this approach is the restrictive representation of individual study variances. Users need to be aware that response bias (heterogeneity) and response heteroscedasticity (over-variability) can each lead to inefficient estimators and hence impaired inference. However, the first step imposed by this method forces the user to reflect carefully on the data sets to be aggregated.

It should be clear to the reader that there are strong connections between our last discussion item involving the linker and the modeling project and this particular topic. The use of the above methodology in SAAM, under the title of multiple studies analysis (Lyne, 1993), required that users link and associate studies manually using arcane and fixed formatted data structures. By contrast, by using the linker in WinSAAM, almost all of the lexical and other manual issues will be automatically coordinated, as the procedure is invoked interactively. Indeed, linked studies for population modeling will emerge as a discrete and highly coordinated function specific to the linker.

Data Exchange with Other Software

Publication preparation, presentation preparation, data preparation and screening, advanced statistical analysis, and advanced graphical management are all activities investigators often need support with but clearly should fall only partly within the scope of kinetic modeling software. We have elected, in designing and implementing WinSAAM, to distribute these function in ways that allow flexibility for users to explore their own array of productivity tools in conjunction with data analysis. Specifically, through opening data exchange channels between WinSAAM and word processors via the clipboard, between WinSAAM and spreadsheet software via the spreadsheet manager, between WinSAAM and graphics processors via either metafile or bitmap encapsulated graphic renditions, and between WinSAAM and advance statistical software via the spreadsheet manager, we have orchestrated absolute flexibility within WinSAAM for leveraging the pertinent processing strengths of related software.

Of course, another way to look at the data exchange issue is to bring a more realistic and manageable closure to the software development and maintenance effort. Rather than duplicating features in other software, if we simply ensure that the opportunity exists to exchange data at an appropriate state with other software in order to facilitate specific processing needs, then we have a clearer and more manageable delineation of the functional boundaries of our evolving system, WinSAAM. We have exercised considerable care in identifying target exchange software and data exchange modes, and at the heart of our consideration have been the basic concerns ... "Is the target software from a setting familiar to the user?" ... "Is the mode of data exchange direct and flexible?" and, to a lesser degree ... "Does the target software have some similarity to and compatibility with the WinSAAM system?"

Third-Party Component Modules

We recognized early in the WinSAAM development project that our ideas could never be realized if all the work were left solely to us. A project of this magnitude calls for multi-faceted skills and contributions well beyond what we were capable of. Accordingly, we designed the system to admit contributions from both the software community and the user community, and also to avoid stalling the system's evolution as these contributions evolved and were themselves incorporated into it.

Specifically we have identified three areas where third-party component modules can be dynamically incorporated into the system:

Functional enhancements to the interface.

Great progress and power enhancement for the user interface has been made possible through our incorporation of third-party software modules. For example, the graphics, spreadsheet, and help utilities have all been refined from commercial tools privately supplied. Other tools which have the potential to enhance the interface power and flexibility will be explored.

Processing enhancement to the kernel.

At the heart of WinSAAM is the SAAM/CONSAM system, encapsulated as a dynamic linking library (DLL). To accomplish this integration, we were required to carefully define the architectural interface between the kernel and the user interface component of the software to ensure that all results from a kernel activation were accessible to the user via the user interface (GUI). Going through this exercise has now provided us with the background, information, and operational details needed to enable us to build on the dynamic linking library concept. As new integrators, optimizers, and allied processing algorithms become available, they will be able to be incorporated into the WinSAAM environment and use repertoire via linking libraries. The two great advantages of pursuing this type of development are, firstly, we can again capitalize on expertise beyond that of the WinSAAM team, and, secondly, we can enhance WinSAAM's capabilities without risking corrupting the kernel code.

Domain Libraries.

It is our belief that the capacity of modeling software such as WinSAAM to support model development constitutes just a small percentage of its return to the scientific community. Considerably greater benefit comes from it being able to deliver the power from the models generated to laboratories, clinics, and educational settings. Indeed, the contributions from investigators such as Cobelli and coworkers have been largely through their unique skill in evolving models, most particularly in the glucose-insulin area (in their case) which address key clinical and allied issues.

In addition to supporting a 'base' model library architecture, WinSAAM will support a comprehensive set of 'domain' libraries which will each include salient models from important application areas. Experts such as Cobelli and his group could be registered as authorized suppliers of a specific domain, in their case glucose-insulin models. Users with access to that domain could then explore the applications of the models encapsulated within it.

Again we have tried to assemble a strategy for extending the application and functionality of WinSAAM by distributing the effort amongst the greater community of biomedical investigators.

Open Architecture

In moving from the CLI (command line interface) style of operation of CONSAM and Mlab (1996) towards the GUI (graphical user interface) style of operation of Stella II (1992) and SAAM II (1997), we are immediately confronted by what are known as 'mode' issues. *Mode-less* software is software that asserts only limited control over the processing flow available for the user. In essence, except for blocking outlandish steps (such as editing a binary file), the mode-less software user is pretty much free to chart her own course. Mode-less software is often described as 'flat,' as the windowing hierarchy is usually fairly shallow.

In contrast to this, *moded* software is somewhat more rigidly controlling in that Windows dispensing services only become available following a particular activation sequence. Another sequence needs to be followed to move hierarchically up through the windows to other tasks and services.

Each type of architecture serves its own niche: moded software tends to be more helpful for use by novices for example, while the mode-less, free-wheeling approach is greatly appreciated by advanced investigators who need to capitalize on their own skills in order to expedite their modeling efforts.

The design of WinSAAM has tended to favor the mode-less approach by providing flexible access to many of its services through a fairly flat architecture. Nevertheless, where necessary and where appropriate or helpful, we have also embraced a moded operation. We feel that a fine balance has to be struck between these diametric provisions. The greater development effort required by a mode-less operation is usually balanced by its commensurate returns to the user, as a consequence of the freedom it allows to explore the ensuing open architecture.

CONCLUSION

We have discussed many of the issues motivating the development of the NIH WinSAAM project. Specifically, we have stressed the care with which we have balanced our development efforts against the expected returns to the user community for that effort. We have identified a series of efficient steps to ensure not only the widest possible participation in this project, but also the highest utility end-product we can possibly evolve.

From the development perspective, and for the user community, *evolve* is the key word. As a very small development team, the only practical way for us to effectively advance the project is by deploying the evolutionary or incremental prototyping paradigm. At the completion of each stage in the evolution of WinSAAM, the next step is negotiated subject to (1) it is subject to incremental incorporation, (2) it demonstrably advances the utility of the software, and (3) its initial implementation is subject to revision against extensive testing and evaluation. Adopting this approach has greatly accelerated the delivery of a 'working' product.

As this chapter was completed, work is underway on WinSAAM's file management and help facilities and, indeed, a first version of the file management facility will only support the integrated WinSAAM input file. Based on the above criteria, we have established a development sequence plan which will see all major components of WinSAAM in place this year.

Because of the current state of the utility of the WinSAAM software and the greater ease with which it fits into the Windows operating environment than either SAAM or Consam, we now invite Consam users to fully participate in the WinSAAM project as registered β-site testers of our new system.

CORRESPONDING AUTHOR

Please address all correspondence to:
Dr. Ray C. Boston
University of Pennsylvania
New Bolton Center
346 West Street Road
Kennettt Square, PA 19348

REFERENCES

ACSL: Advanced Continuous Simulation Language, Version 10.2. Mitchell and Gauthier Asscociates: Concord, MA. 1992.

Baldwin RL; France J; Beever DE; Gill M; Thornley JHM. Metabolism of the Lactating Cow III: Properties of Mechanistic Models Suitable for Evaluation of Energetic Relationships and Factors Involved in the Partition of Nutrients. *J Dairy Res,* 1987, 54:133-145.

Baldwin RL. *Modeling Ruminant Digestion and Metabolism.* Chapman Hall:London. 1995.

Berman M; Weiss MF. *SAAM 27 Manual.* DHEW Publ. (NIH) 78-180. Laboratory of Theoretical Biology, NCI, NIH: Bethesda, MD. 1963-1978.

Berman M; Grundy SM; Howard BV. *Lipoprotein Kinetics and Modeling.* Academic Press: New York. 1982.

Boston RC; Greif PC; Berman M. Conversational SAAM-an interactive program for the kinetic analysis of biological systems. *Comput Prog Biomed,* 1981, 13:111-119.

Boston RC; Weber KM. Modeling with SAAM and its advancement in association with the study of mineral metabolism. *Math Biosci,* 1984, 72(2):181-198.

Carson ER; Cobelli C; Finkelstein L. *The Mathematical Modeling of Metabolic and Endocrine Systems.* John Wiley: New York. 1983.

Diggle PJ; Liang KV; Zeger SL. *Analysis of Longitudinal Data.* Oxford University Press: Oxford. 1994.

Dijkstra J; Neal HDStC; Beever DE; France J. Simulation of nutrient digestion, absorption and outflow in the rumen: Model description. *J Nutr,* 1992, 122:2239-2256.

Dijkstra J. *Mathematical Modeling and Integration of Rumen Fermentation Processes.* Univ. Wageningen Press: Netherlands. 1993.

Faichney GJ; Boston RC. Movement of water within the body of sheep fed at maintenance under thermoneutral conditions. *Aust J Biol Sci,* 1985, 38:85-94.

France J; Thornley JHM; Beever DE. A mathematical model of the rumen. *J Agric Sci Camb,* 1982, 99:343-353.

France J; Thornley JHM. *Mathematical Models in Agriculture.* Butterworths: London. 1984.

Fugazi K. *Continuous System Modeling for the B6700.* Computer Center: University of California, Davis, CA. 1974.

Gill M; Thornley JHM; Black JL; Oldham JD; Beever DE. Simulation of the metabolism of absorbed energy-yielding nutrients in young sheep. *B J Nutr,* 1984, 52:621-649.

Hanigan MD; Baldwin RL. A mechanistic model of mammary gland metabolism in the lactating cow. *Agric Systems,* 1994, 45:369-393.

Jacquez JA. *Compartmental Analysis in Biology and Medicine.* Biomedware: Ann Arbor. 1996.

Lyne A; Boston R; Pettigrew K; Zech L. EMSA – A SAAM service for the estimation of population parameters based on model fits to identically replicated experiments. *Comp Mthds Programs Biomed,* 1992, 38:117-151.

Mlab: A Mathematical Laboratory. Civilized Software, Inc.: Bethesda, MD. Revision Date: Oct. 8, 1996.

Neal HDStC; Thornley JHM. The lactation curve in cattle: A mathematical model of the mammary gland. *J Agric Sci Camb,* 1983, 101:389-400.

Pettigrew FE; Gill M; France J; Close WH. A mathematical integration of energy and amino acid metabolism of lactating sows. *J Ani Sci,* 1992, 70:3742-3761.

Pressman RS. *Software Engineering: A Practioner's Approach.* McGraw Hill: New York. 1982.

SAAM II: A Program for Kinetic Modeling. Resource Facility for Kinetic Analysis: University of Washington, Seattle, WA. 1997.

Scientist: For Experimental Data Fitting, Version 2.0. MicroMath Scientific Software: Salt Lake City, Utah. 1995.

STATA: Release 5.0, Statistics, Data Management, Graphics. StataCorp: College Station, TX. 1997.

Stella II: An Introduction to Systems Thinking. High Performance Systems, Inc.: Hanover, NH. 1992.

Thornley JHM; Johnson IR. *Plant and Crop Modeling: A Mathematical Approach to Plant and Crop Physiology.* Oxford University Press: Oxford. 1990.